

Experiment – 1.1.1

Student Name: Himanshu Goyal

UID: 24BDA70369

Branch: CSE (Data Science)

Section/Group: 24 BDS-4(B)

Semester: 5th

Date of Performance: 25-07-2026

Subject: Full Stack II

Subject Code: 24CSP-337

1. Aim:

To design and develop a dynamic post composer interface supporting multiple platforms with constraint validation.

2. Software Required:

- IDE/Environment: Visual Studio Code
- Runtime: Node.js (v18 or above), npm
- Framework/Libraries: React 18, Vite, Redux Toolkit, react-redux
- Browser: Google Chrome (for testing and rendering)

3. Objective:

The UI should allow users to input textual content, attach media, and select one or more publishing platforms. Based on platform selection, real-time validation must be enforced to ensure compliance with constraints such as character limits and hashtag rules. Visual feedback mechanisms such as character counters, warnings, and error indicators enhance the user experience. The system should be modular, reusable, and responsive.

4. Theory:

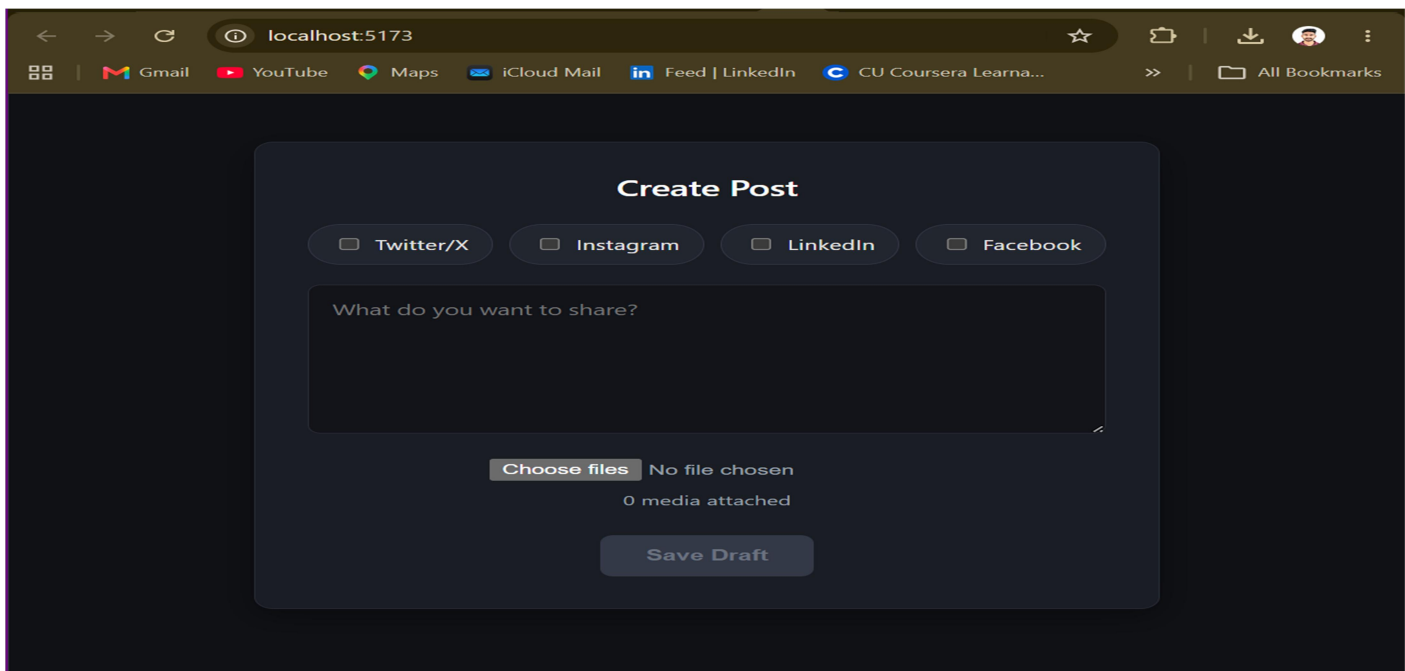
React functional components with Hooks (useState, useMemo, useCallback) allow a UI to derive and recompute values only when their dependencies change. In this experiment, platform metadata (character limits, media limits) is centralized in a Redux store rather than hardcoded per-platform in the component, so the same validation function can be reused across all platforms. Controlled components (textarea bound to state) allow the UI to react instantly to user input, enabling real-time validation feedback.

5. Procedure/Algorithm:

- Step 1: Define platform metadata (name, character limit, max media) in the Redux store.
- Step 2: Render a checkbox chip for each platform; toggling adds/removes it from the selected set.
- Step 3: Bind a textarea to local state (text) and a file input to a media array.
- Step 4: For each selected platform, run a validateForPlatform() function checking character count, media count, and platform-specific rules.
- Step 5: Display per-platform feedback, and drive a live progress bar/counter from the strictest selected platform's limit.

6. Code and Output:

```
function validateForPlatform(text, media, platform) {  
  const errors = [];  
  if (text.length > platform.charLimit) {  
    errors.push(`${text.length - platform.charLimit} characters over limit`);  
  }  
  if (media.length > platform.maxMedia) {  
    errors.push(`${media.length} media items exceeds max`);  
  }  
  return errors;  
}
```



7. Conclusion:

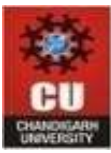
A single composer screen was successfully built that dynamically adapts its validation and feedback to whichever combination of platforms is selected, without platform-specific branching in the JSX.

8. Learning Outcomes:

- Learned to build controlled React components driven entirely by state.
- Understood how to derive real-time validation feedback using useMemo.
- Learned to design a UI that is data-driven rather than hardcoded per use case.

Evaluation Grid (To be created as per the SOP and Assessment guidelines by the faculty):

Sr. No.	Parameters	Marks Obtained	Maximum Marks
1.	Worksheet completion		08
2.	Post Lab Quiz Result/Viva-Voce		10
3.	Conduct		12
	Total		30



Experiment – 1.1.2

Student Name: Himanshu Goyal

UID: 24BDA70369

Branch: CSE (Data Science)

Section/Group: 24 BDS-4(B)

Semester: 5th

Date of Performance: 25-07-2026

Subject: Full Stack II

Subject Code: 24CSP-337

1. Aim:

To implement a draft management system that allows users to save, retrieve, and manage post drafts within the frontend, with optional simulation of backend interactions.

2. Software Required:

- IDE/Environment: Visual Studio Code
- Runtime: Node.js (v18 or above), npm
- Framework/Libraries: React 18, Vite, Redux Toolkit
- Browser: Google Chrome (for testing and rendering)

3. Objective:

Users should be able to save an in-progress post as a draft, retrieve any saved draft back into the composer for further editing, and delete drafts they no longer need — all persisted in frontend state (Redux). The system should optionally simulate the latency of a real backend save/fetch operation.

4. Theory:

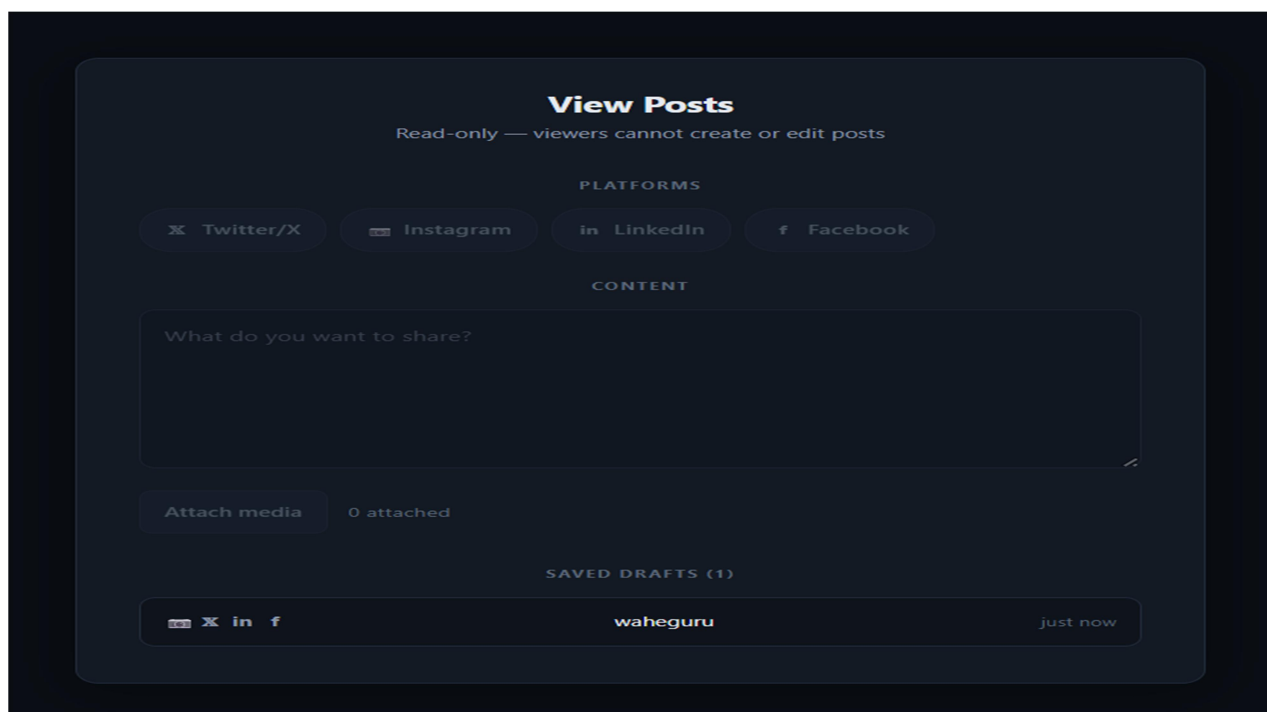
A draft management system needs three operations — create, update, and delete — each represented as a Redux action. Storing drafts in normalized form (`{ byId, allIds }`) means loading a draft back into the editor, or deleting one, is an $O(1)$ lookup. "Simulating backend interaction" refers to wrapping a save/delete action in an async thunk with an artificial delay, so the UI can show a pending state exactly as it would when waiting on a real network request.

5. Procedure/Algorithm:

- Step 1: Define draftCreated, draftUpdated, and draftDeleted reducers in the Redux slice.
- Step 2: On Save Draft, dispatch draftCreated (first save) or draftUpdated (subsequent saves).
- Step 3: Render a Saved Drafts list from the store, showing a text preview and relative timestamp.
- Step 4: On clicking a draft, load its stored fields back into the composer's local state.
- Step 5: On clicking delete, dispatch draftDeleted and remove the draft from state.

6. Code and Output:

```
draftCreated: {  
  reducer(state, action) {  
    const draft = action.payload;  
    state.drafts.byId[draft.id] = draft;  
    state.drafts.allIds.push(draft.id);  
  },  
  prepare({ text, platformIds, media }) {  
    return { payload: { id: nanoid(), text, platformIds, media,  
      createdAt: Date.now(), updatedAt: Date.now() } };  
  },  
},  
draftDeleted(state, action) {  
  const id = action.payload;  
  delete state.drafts.byId[id];  
  state.drafts.allIds = state.drafts.allIds.filter((d) => d !== id);  
},
```



7. Conclusion:

A complete draft lifecycle was implemented: users can save a post in progress, see it appear instantly in a Saved Drafts list, click it to resume editing, and delete it when no longer needed — all backed by normalized Redux state.

8. Learning Outcomes:

- Learned to model a full CRUD lifecycle for a frontend-only entity using Redux Toolkit.
- Understood how normalized state keeps draft operations at $O(1)$.
- Learned to simulate asynchronous backend latency with `createAsyncThunk`.

Evaluation Grid (To be created as per the SOP and Assessment guidelines by the faculty):

Sr. No.	Parameters	Marks Obtained	Maximum Marks
1.	Worksheet completion		08
2.	Post Lab Quiz Result/Viva-Voce		10
3.	Conduct		12
	Total		30

Experiment – 1.2.1

Student Name: Himanshu Goyal

UID: 24BDA70369

Branch: CSE (Data Science)

Section/Group: 24 BDS-4(B)

Semester: 5th

Date of Performance: 25-07-2026

Subject: Full Stack II

Subject Code: 24CSP-337

1. Aim:

To design and implement a centralized state management system using Redux Toolkit for managing posts and platform-related data.

2. Software Required:

- IDE/Environment: Visual Studio Code
- Runtime: Node.js (v18 or above), npm
- Framework/Libraries: React 18, Vite, Redux Toolkit, react-redux
- Browser: Google Chrome (for testing and rendering)

3. Objective:

This experiment introduces the concept of global state, state normalization, and structured state design to improve scalability, maintainability, and performance of frontend applications.

4. Theory:

Redux Toolkit's createSlice combines action creators and reducers, and uses Immer internally so reducers can be written as direct state mutations while remaining immutable under the hood. Normalized state — storing entities as { byId, allIds } instead of a plain array — gives O(1) lookup and update by id.

5. Procedure/Algorithm:

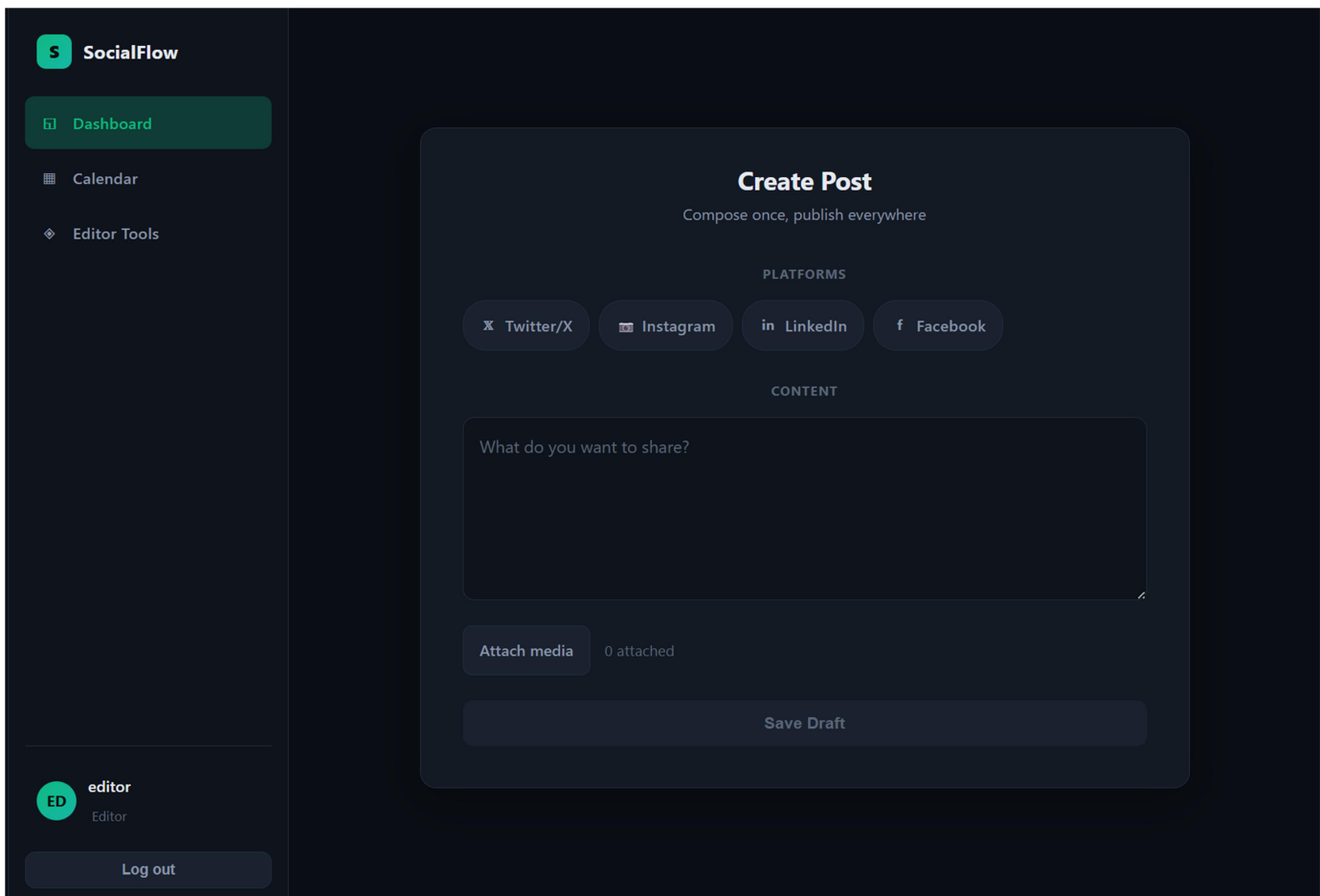
- Step 1: Define initialState with three normalized entities: posts, drafts, platforms.
- Step 2: Pre-populate platform metadata with their constraints.
- Step 3: Write reducers for draft/post lifecycle actions.
- Step 4: Register the slice reducer inside configureStore.

- Step 5: Wrap the app in `<Provider store={store}>`.

6. Code and Output:

```
const initialState = {
  posts: { byId: {}, allIds: [] },
  drafts: { byId: {}, allIds: [] },
  platforms: {
    byId: { twitter: { id: 'twitter', name: 'Twitter/X', charLimit: 280, maxMedia: 4 } },
    allIds: ['twitter', 'instagram', 'linkedin', 'facebook'],
  },
};

export const store = configureStore({ reducer: { posts: postsReducer, auth: authReducer } });
```



7. Conclusion:

A single, predictable source of truth was established for posts, drafts, and platform configuration, confirmed by dispatching actions from the UI and observing state update immediately across all connected components.

8. Learning Outcomes:

- Learned to design normalized state shape for scalable entity management.
- Understood how Redux Toolkit's Immer integration simplifies reducer code.
- Learned to structure a slice covering multiple related entities in one store.

Evaluation Grid (To be created as per the SOP and Assessment guidelines by the faculty):

Sr. No.	Parameters	Marks Obtained	Maximum Marks
1.	Worksheet completion		08
2.	Post Lab Quiz Result/Viva-Voce		10
3.	Conduct		12
	Total		30

Experiment – 1.2.2

Student Name: Himanshu Goyal

UID: 24BDA70369

Branch: CSE (Data Science)

Section/Group: 24 BDS-4(B)

Semester: 5th

Date of Performance: 25-07-2026

Subject: Full Stack II

Subject Code: 24CSP-337

1. Aim:

To optimize state access and improve application performance using memoized selectors and efficient rendering strategies.

2. Software Required:

- IDE/Environment: Visual Studio Code
- Runtime: Node.js (v18 or above), npm
- Framework/Libraries: React 18, Vite, Redux Toolkit (createSelector)
- Browser: Google Chrome (for testing and rendering)

3. Objective:

This experiment introduces concepts such as derived state, selector optimization, and minimizing unnecessary re-renders to ensure high performance in complex frontend applications.

4. Theory:

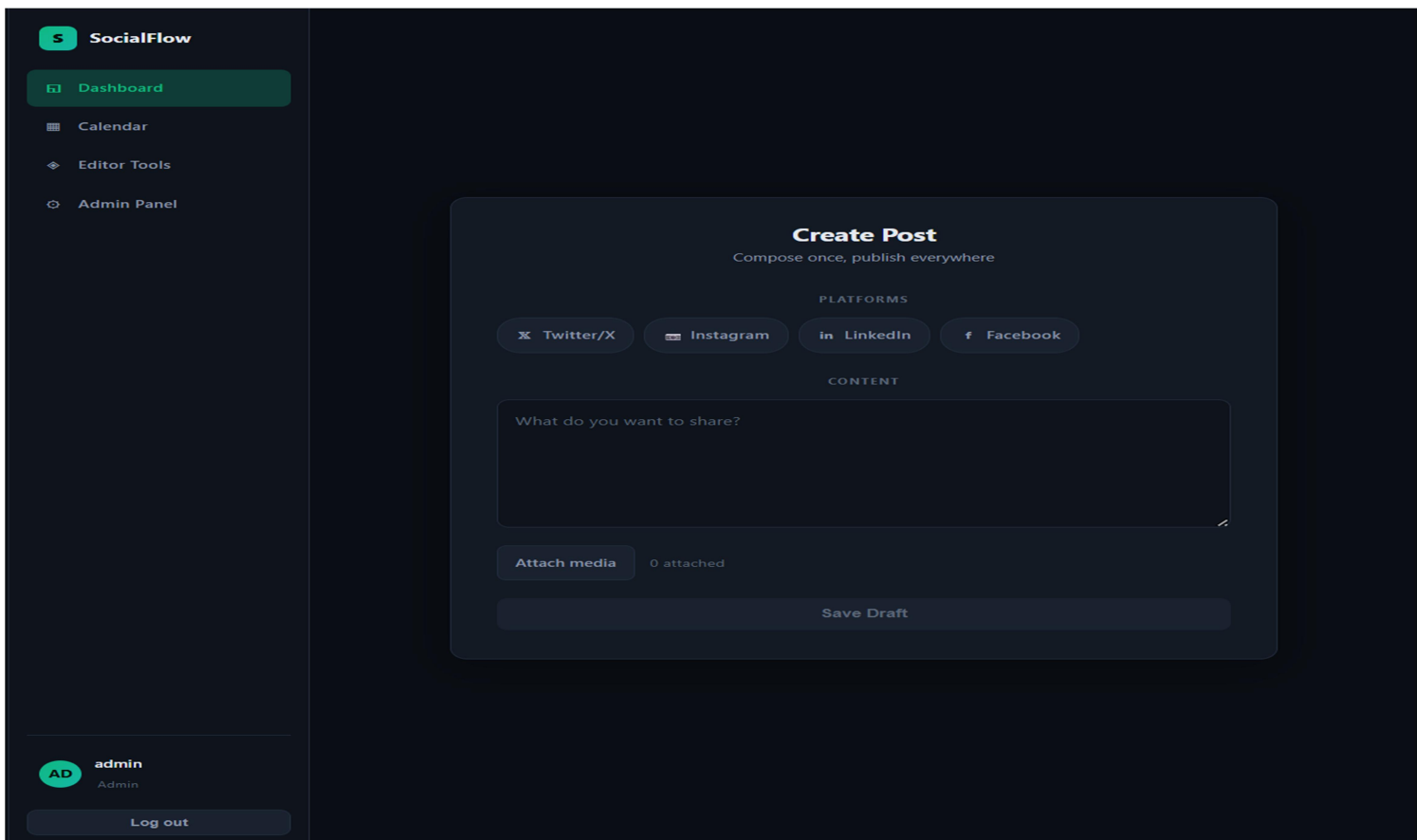
Every call to `useSelector` re-runs the selector function on every dispatched action. If a selector returns a newly created array or object each time, React-Redux sees a new reference and re-renders even if data is unchanged. Wrapping derived selectors in `createSelector` caches the output by input reference.

5. Procedure/Algorithm:

- Step 1: Identify all places in the UI where data is filtered, mapped, or aggregated.
- Step 2: Replace inline component-level filtering with a dedicated `selectors.js` module.
- Step 3: Wrap each derived selector in `createSelector`.
- Step 4: Build compound selectors from base selectors rather than recomputing from raw state.

6. Code and Output:

```
export const selectAllDrafts = createSelector(  
  [selectDraftsState],  
  (drafts) => drafts.allIds.map((id) => drafts.byId[id])  
);  
  
export const selectDraftCountByPlatform = createSelector(  
  [selectAllDrafts, selectAllPlatforms],  
  (drafts, platforms) => {  
    const counts = Object.fromEntries(platforms.map((p) => [p.id, 0]));  
    for (const d of drafts) for (const pid of d.platformIds) counts[pid]++;  
    return counts;  
  }  
);
```



7. Conclusion:

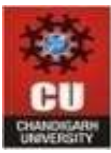
By routing every derived value through createSelector, the application avoids unnecessary recomputation and re-renders, keeping the UI responsive as the number of drafts and posts grows.

8. Learning Outcomes:

- Understood how referential equality affects React-Redux re-renders.
- Learned to build compound memoized selectors from simpler base selectors.
- Learned to identify and eliminate inline derived-state computation inside components.

Evaluation Grid (To be created as per the SOP and Assessment guidelines by the faculty):

Sr. No.	Parameters	Marks Obtained	Maximum Marks
1.	Worksheet completion		08
2.	Post Lab Quiz Result/Viva-Voce		10
3.	Conduct		12
	Total		30



Experiment – 1.3.1

Student Name: Himanshu Goyal

UID: 24BDA70369

Branch: CSE (Data Science)

Section/Group: 24 BDS-4(B)

Semester: 5th

Date of Performance: 25-07-2026

Subject: Full Stack II

Subject Code: 24CSP-337

1. Aim:

To design and implement a secure authentication system using JWT for user login and session management.

2. Software Required:

- IDE/Environment: Visual Studio Code
- Runtime: Node.js (v18 or above), npm
- Backend: Express.js, jsonwebtoken, bcryptjs, cors
- Frontend: React 18, Vite, Redux Toolkit, react-redux
- Browser: Google Chrome (for testing and rendering)

3. Objective:

Passwords must never be stored or compared in plain text; the server should issue a signed, time-limited token on successful login, and the client should persist and revalidate that token across page reloads.

4. Theory:

A JSON Web Token is a compact, digitally signed string. The server signs the token with a secret key (jwt.sign); requests present it in an Authorization: Bearer header, verified via jwt.verify. Passwords are hashed with bcrypt (per-password salt) and compared via bcrypt.compareSync, never in plain text.

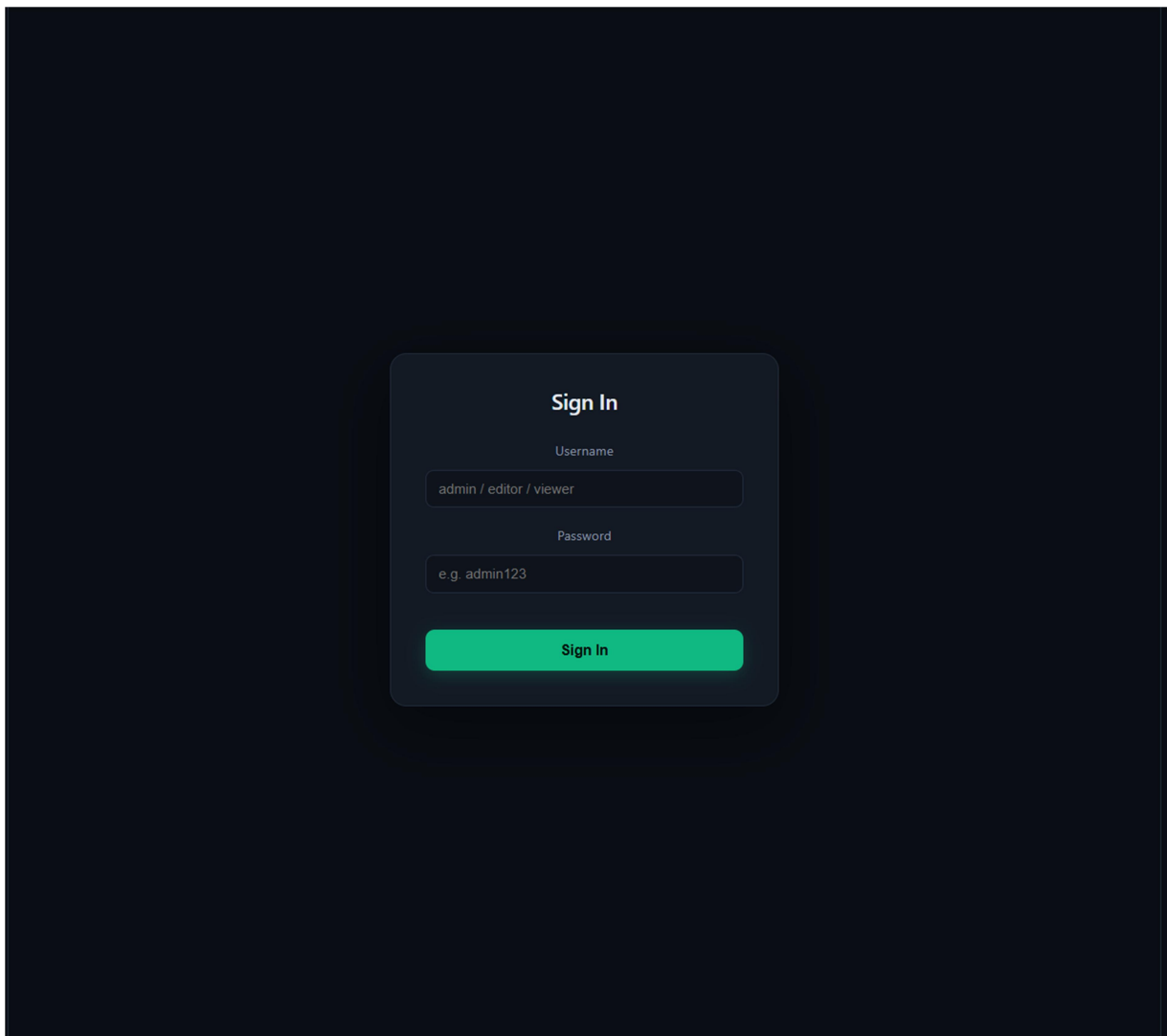
5. Procedure/Algorithm:

- Step 1: Store users with bcrypt-hashed passwords and an assigned role.
- Step 2: On POST /api/login, verify credentials and reject with 401 on failure.
- Step 3: On success, sign a JWT with a 15-minute expiry.
- Step 4: Write authenticate middleware to verify the token on protected routes.

- Step 5: Build an authSlice with login/checkSession thunks, persisting the token in localStorage.

6. Code and Output:

```
app.post('/api/login', (req, res) => {  
  const user = users.find((u) => u.username === req.body.username);  
  if (!user || !bcrypt.compareSync(req.body.password, user.passwordHash)) {  
    return res.status(401).json({ message: 'Invalid username or password' });  
  }  
  const token = jwt.sign(  
    { id: user.id, username: user.username, role: user.role },  
    JWT_SECRET, { expiresIn: '15m' }  
  );  
  res.json({ token, user: { id: user.id, username: user.username, role: user.role } });  
});
```



The image shows a dark-themed login form titled "Sign In". It has two input fields: "Username" with the placeholder text "admin / editor / viewer" and "Password" with the placeholder text "e.g. admin123". Below the fields is a red "Sign In" button. The form is centered on a dark background.

7. Conclusion:

A working authentication system was implemented where passwords are hashed with bcrypt and a signed, time-limited JWT is issued and verified on every request. The session persists across page reloads via localStorage and server revalidation.

8. Learning Outcomes:

- Learned why passwords must be hashed rather than stored in plain text.
- Understood JWT structure and why its payload must never contain sensitive data.
- Learned to persist and revalidate a session using localStorage and a session-check API call.

Evaluation Grid (To be created as per the SOP and Assessment guidelines by the faculty):

Sr. No.	Parameters	Marks Obtained	Maximum Marks
1.	Worksheet completion		08
2.	Post Lab Quiz Result/Viva-Voce		10
3.	Conduct		12
	Total		30

Experiment – 1.3.2

Student Name: Himanshu Goyal

UID: 24BDA70369

Branch: CSE (Data Science)

Section/Group: 24 BDS-4(B)

Semester: 5th

Date of Performance: 25-07-2026

Subject: Full Stack II

Subject Code: 24CSP-337

1. Aim:

To implement role-based access control and secure application routes based on user permissions.

2. Software Required:

- IDE/Environment: Visual Studio Code
- Runtime: Node.js (v18 or above), npm
- Backend: Express.js, jsonwebtoken
- Frontend: React 18, Vite, Redux Toolkit, react-redux, react-router-dom
- Browser: Google Chrome (for testing and rendering)

3. Objective:

Different user roles (admin, editor, viewer) should see and access different parts of the application, enforced both in the UI and at the API level.

4. Theory:

Role-based access control embeds a role claim inside the JWT and checks it at two layers: server middleware (requireRole) rejecting disallowed requests with 403, and client route guards redirecting unauthorized users before rendering restricted content.

5. Procedure/Algorithm:

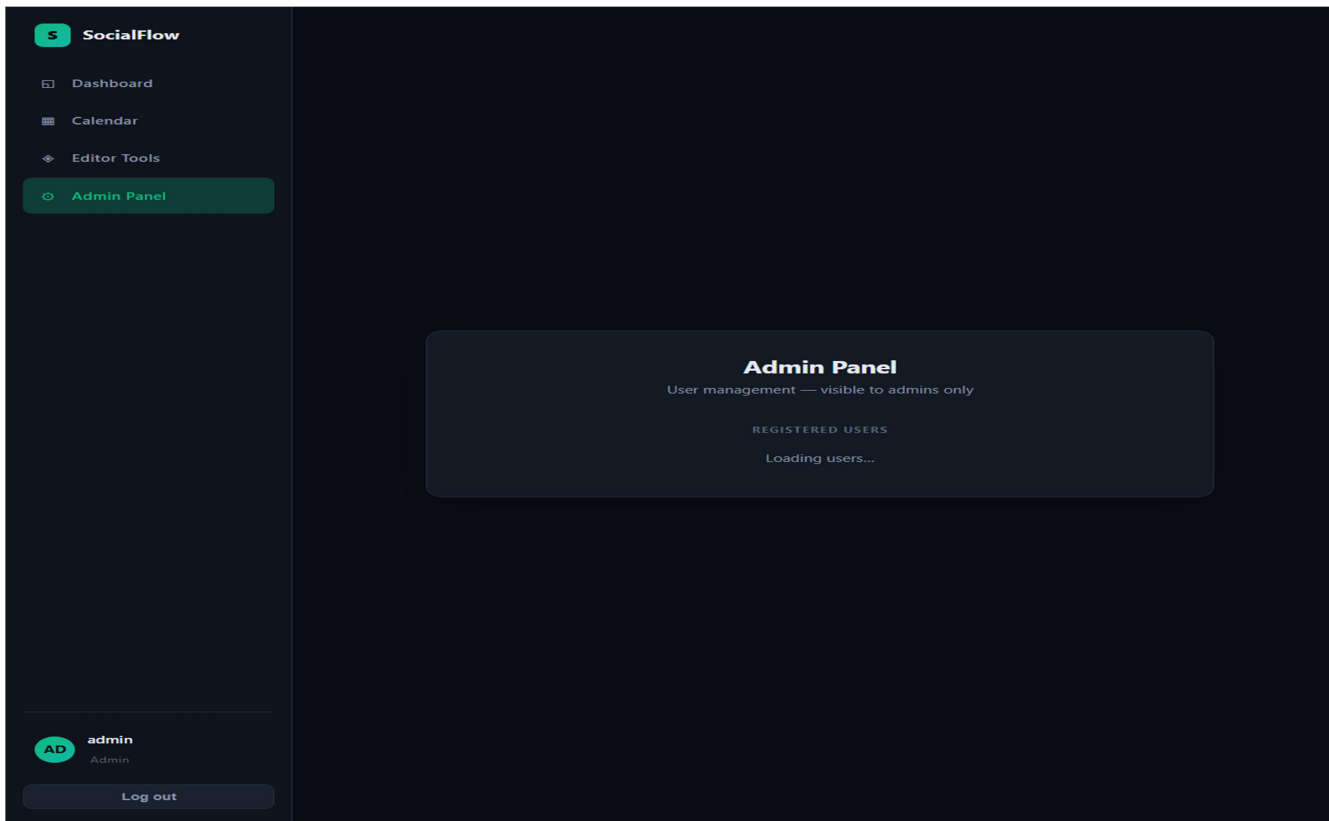
- Step 1: Assign each user a role and embed it in the JWT payload at login.
- Step 2: Write requireRole(...allowedRoles) middleware checking req.user.role.
- Step 3: Apply requireRole to admin-only backend routes.
- Step 4: Build a ProtectedRoute component redirecting based on role.

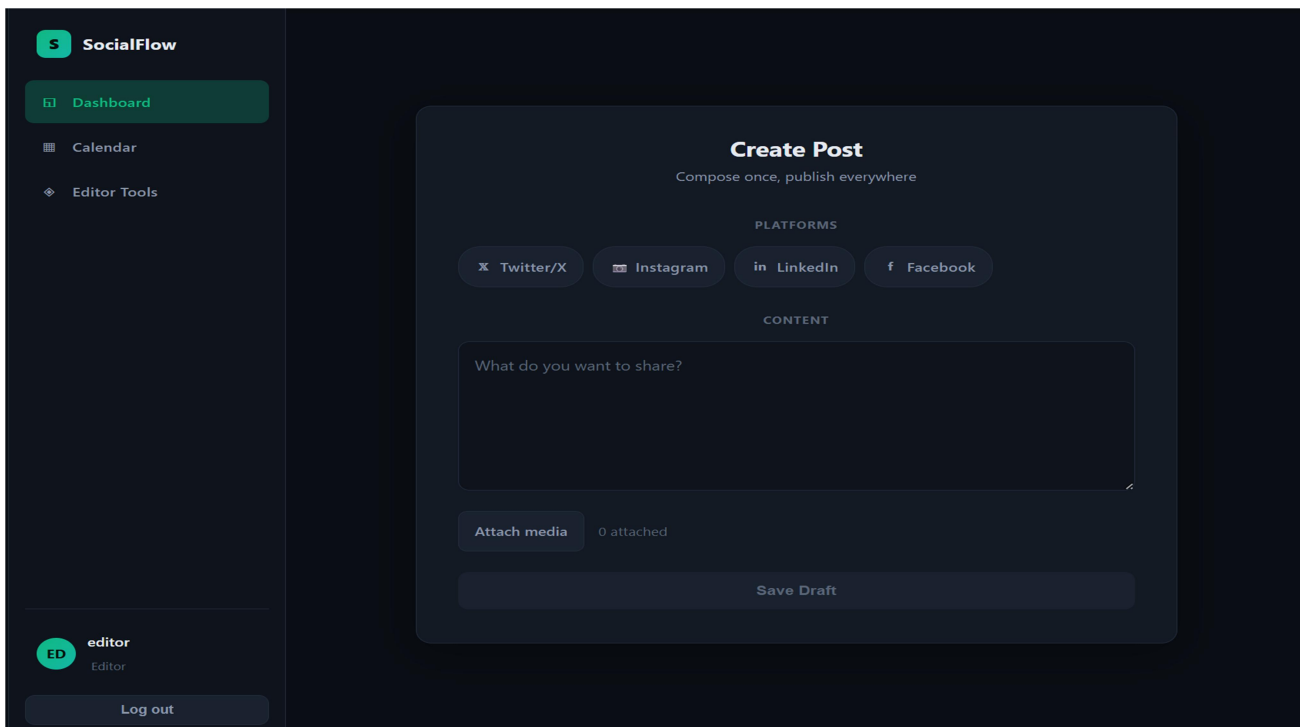
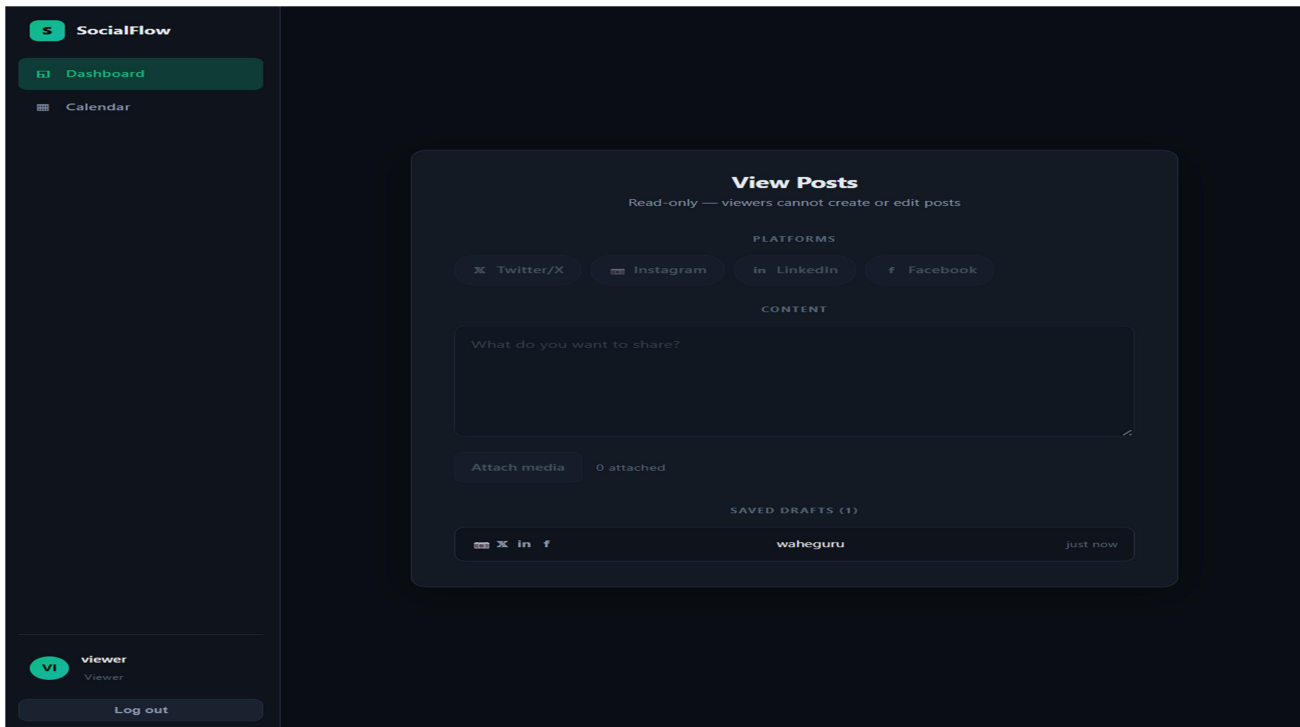
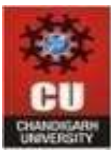
- Step 5: Conditionally render sidebar links and page content per role.

6. Code and Output:

```
function requireRole(...allowedRoles) {
  return (req, res, next) => {
    if (!req.user || !allowedRoles.includes(req.user.role)) {
      return res.status(403).json({ message: 'Forbidden: insufficient role' });
    }
    next();
  };
}

app.get('/api/admin/users', authenticate, requireRole('admin'), (req, res) => {
  res.json({ users: users.map((u) => ({ id: u.id, username: u.username, role: u.role }))) });
});
```





7. Conclusion:

Role-based access control was successfully layered on top of authentication: admin, editor, and viewer accounts each see a different sidebar and have different route access, enforced at both server and client layers — confirmed by the admin's full navigation versus the viewer's restricted, read-only Dashboard.

8. Learning Outcomes:

- Learned to write reusable Express middleware for role-based authorization.
- Learned to enforce access control at two independent layers.
- Understood how to build memoized, parameterized selectors for role checks.

Evaluation Grid (To be created as per the SOP and Assessment guidelines by the faculty):

Sr. No.	Parameters	Marks Obtained	Maximum Marks
1.	Worksheet completion		08
2.	Post Lab Quiz Result/Viva-Voce		10
3.	Conduct		12
	Total		30

Experiment – 1.4.1

Student Name: Himanshu Goyal

UID: 24BDA70369

Branch: CSE (Data Science)

Section/Group: 24 BDS-4(B)

Semester: 5th

Date of Performance: 25-07-2026

Subject: Full Stack II

Subject Code: 24CSP-337

1. Aim:

To design and implement an interactive calendar interface for scheduling and managing posts.

2. Software Required:

- IDE/Environment: Visual Studio Code
- Runtime: Node.js (v18 or above), npm
- Framework/Libraries: React 18, Vite, Redux Toolkit
- Browser: Google Chrome (for testing and rendering)

3. Objective:

This experiment introduces temporal data visualization, event mapping, user interaction patterns (drag-and-drop), and efficient rendering of time-based data within frontend applications.

4. Theory:

A month grid is built from plain Date arithmetic — computing the first weekday of the month and the number of days to lay out cells correctly. Event mapping groups scheduled items by date via a memoized selector, so the calendar renders only the events relevant to each cell. Native HTML5 drag-and-drop (dataTransfer, onDragStart/onDragOver/onDrop) provides the interaction pattern without needing an external library.

5. Procedure/Algorithm:

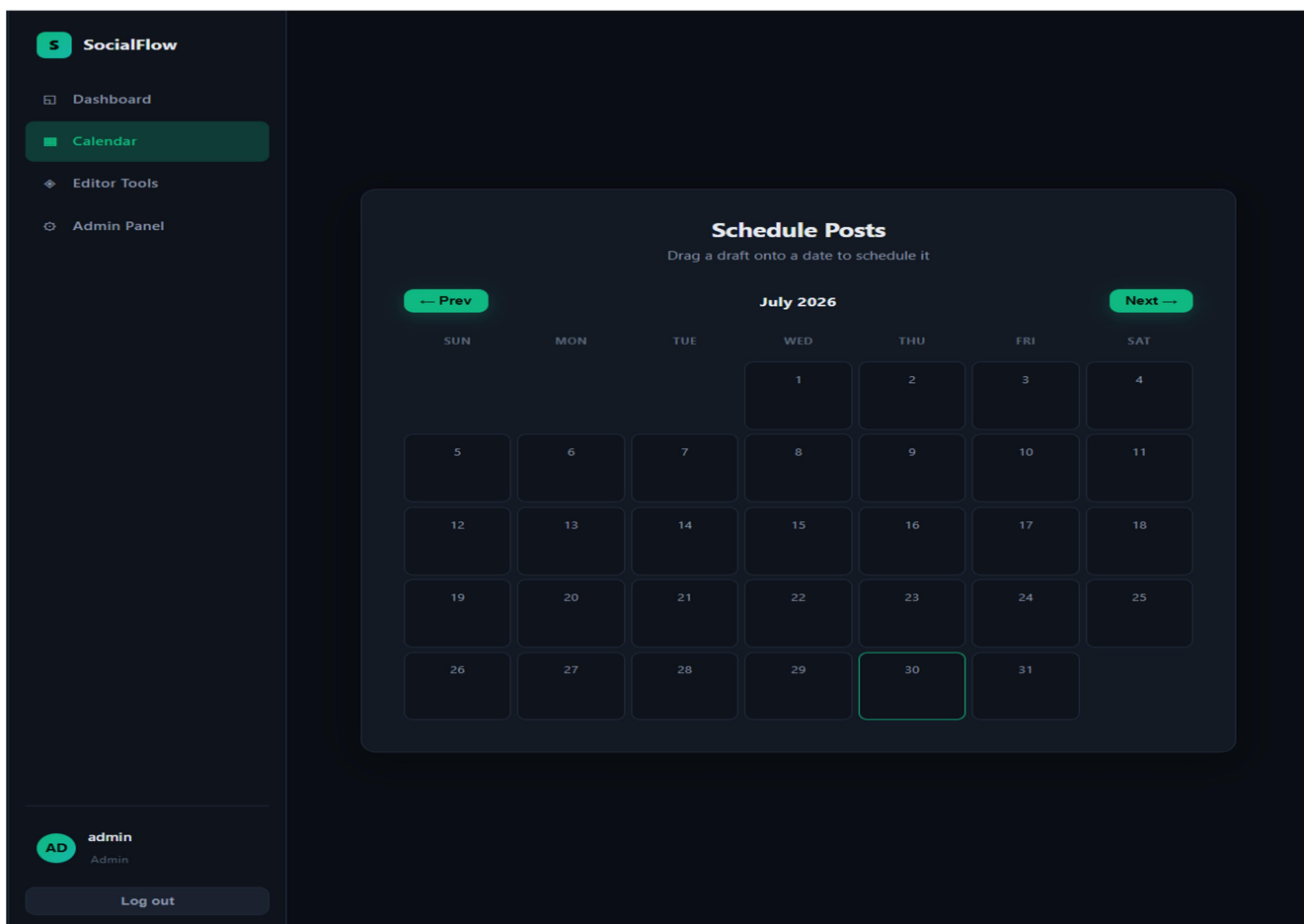
- Step 1: Build a month grid array from Date math, including leading blanks for correct weekday alignment.
- Step 2: Group scheduled drafts by date using a memoized selector (selectDraftsByScheduledDate).

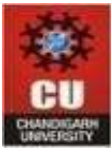
- Step 3: Render each day as a drop target (onDragOver, onDrop) and each draft as a draggable card (draggable, onDragStart).
- Step 4: On drop, dispatch draftScheduled({ id, scheduledAt }) to update the draft's date in Redux.
- Step 5: Maintain an "Unscheduled Drafts" pool for drafts with no scheduledAt yet.

6. Code and Output:

```
function buildMonthGrid(year, month) {
  const startDay = new Date(year, month, 1).getDay();
  const daysInMonth = new Date(year, month + 1, 0).getDate();
  const cells = [];
  for (let i = 0; i < startDay; i++) cells.push(null);
  for (let d = 1; d <= daysInMonth; d++) cells.push(new Date(year, month, d));
  return cells;
}

const handleDrop = (e, dateKey) => {
  e.preventDefault();
  const draftId = e.dataTransfer.getData('text/draft-id');
  dispatch(draftScheduled({ id: draftId, scheduledAt: dateKey }));
};
```





S SocialFlow

Dashboard

Calendar

Schedule Posts

Drag a draft onto a date to schedule it

← Prev

July 2026

Next →

SUN

MON

TUE

WED

THU

FRI

SAT

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

    waheguru

26

27

28

29

30

31

VI viewer

Viewer

Log out

7. Conclusion:

An interactive month-view calendar was implemented where drafts can be dragged from an unscheduled pool onto any date, immediately updating their scheduledAt field in Redux and re-rendering the correct day cell — confirming the event-mapping and drag-and-drop interaction pattern works end-to-end.

8. Learning Outcomes:

- Learned to build a calendar grid from raw Date arithmetic without a charting/calendar library.
- Learned to implement native HTML5 drag-and-drop without external dependencies.
- Understood how to map time-based events to a grid using a memoized grouping selector.

Evaluation Grid (To be created as per the SOP and Assessment guidelines by the faculty):

Sr. No.	Parameters	Marks Obtained	Maximum Marks
1.	Worksheet completion		08
2.	Post Lab Quiz Result/Viva-Voce		10
3.	Conduct		12
	Total		30

Experiment – 1.4.2

Student Name: Himanshu Goyal

UID: 24BDA70369

Branch: CSE (Data Science)

Section/Group: 24 BDS-4(B)

Semester: 5th

Date of Performance: 25-07-2026

Subject: Full Stack II

Subject Code: 24CSP-337

1. Aim:

To optimize rendering performance and implement testing strategies for interactive UI components.

2. Software Required:

- IDE/Environment: Visual Studio Code
- Runtime: Node.js (v18 or above), npm
- Testing: Vitest, @testing-library/react, @testing-library/jest-dom, jsdom
- Browser: Google Chrome (for testing and rendering)

3. Objective:

Interactive components such as the drag-and-drop calendar must avoid unnecessary re-renders during high-frequency events (e.g. drag-over), and the application's core logic — validation, selectors, and CRUD actions — must be verified with an automated test suite rather than relying solely on manual testing.

4. Theory:

React.memo prevents a component from re-rendering when its props haven't changed, which matters for components inside a parent that re-renders frequently — such as calendar day cells during a drag operation, where the parent's dragOverDate state changes on nearly every pixel of mouse movement. For testing, React Testing Library renders components into a virtual DOM (jsdom) and lets tests interact with them the way a user would (fireEvent.click, fireEvent.change), asserting on rendered output rather than internal implementation details. Testing memoized selectors specifically requires checking referential equality (toBe), not just value equality, to prove memoization is actually happening.

5. Procedure/Algorithm:

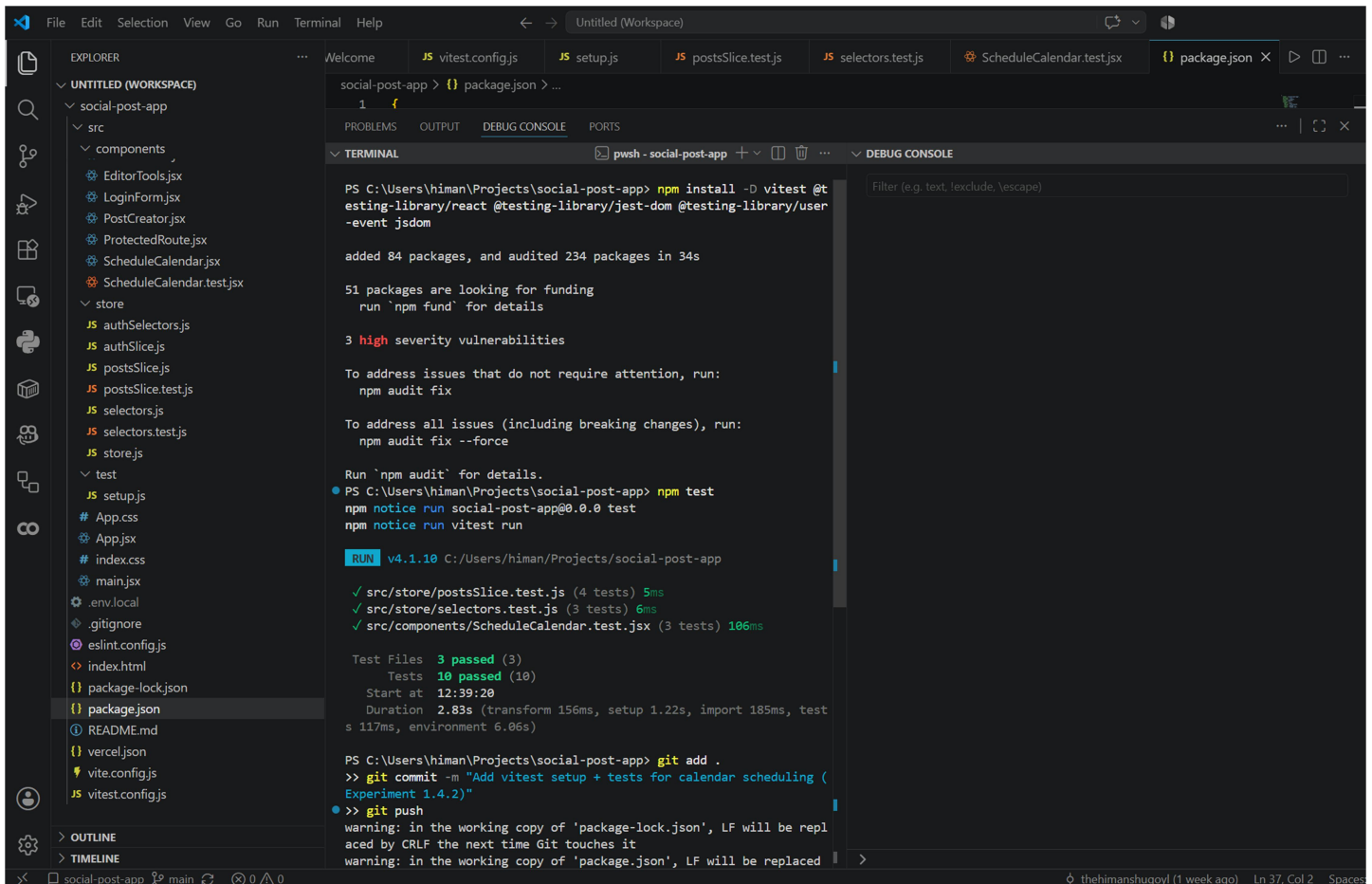
- Step 1: Identify the highest-frequency re-render source in the interactive component (drag-over state changes in the calendar).
- Step 2: Extract the repeated child (day cell) into its own component and wrap it in React.memo.
- Step 3: Wrap all event handlers passed as props in useCallback so their reference stays stable, allowing memo to actually skip re-renders.
- Step 4: Install Vitest and React Testing Library; configure jsdom as the test environment.
- Step 5: Write selector tests asserting both correct output and referential equality across repeated calls.
- Step 6: Write component tests simulating real user interaction (click, type, submit) and asserting on rendered DOM.

6. Code and Output:

```
// Performance: memoized day cell avoids re-rendering all 35+ cells on every drag event
const DayCell = memo(function DayCell({ date, drafts, isDragOver, ...handlers }) {
  /* ... */
});

// Testing: verifying memoization via referential equality, not just output value
it('is memoized: same state reference returns same array reference', () => {
  const state = makeState([{ id: 'a', text: 'x', platformIds: [], media: [] }]);
  const first = selectAllDrafts(state);
  const second = selectAllDrafts(state);
  expect(first).toBe(second);
});

it('enables Save Draft once a platform is selected and text is within limits', () => {
  renderWithStore(<PostCreator />);
  fireEvent.click(screen.getByText('Twitter/X'));
  fireEvent.change(screen.getByPlaceholderText(/what do you want to share/i), {
    target: { value: 'short post' },
  });
  expect(screen.getByRole('button', { name: /save draft/i })).not.toBeDisabled();
});
```



```

social-post-app > {} package.json > ...
1 f
PROBLEMS OUTPUT DEBUG CONSOLE PORTS
TERMINAL
pwsh - social-post-app + v ... DEBUG CONSOLE
Filter (e.g. text, exclude, \escape)

PS C:\Users\himan\Projects\social-post-app> npm install -D vitest @testing-library/react @testing-library/jest-dom @testing-library/user-event jsdom

added 84 packages, and audited 234 packages in 34s

51 packages are looking for funding
  run `npm fund` for details

3 high severity vulnerabilities

To address issues that do not require attention, run:
  npm audit fix

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
● PS C:\Users\himan\Projects\social-post-app> npm test
npm notice run social-post-app@0.0.0 test
npm notice run vitest run

RUN v4.1.10 C:\Users\himan\Projects\social-post-app

✓ src/store/postsSlice.test.js (4 tests) 5ms
✓ src/store/selectors.test.js (3 tests) 6ms
✓ src/components/ScheduleCalendar.test.jsx (3 tests) 106ms

Test Files 3 passed (3)
Tests 10 passed (10)
Start at 12:39:20
Duration 2.83s (transform 156ms, setup 1.22s, import 185ms, test 117ms, environment 6.06s)

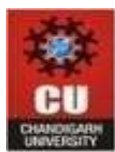
PS C:\Users\himan\Projects\social-post-app> git add .
>> git commit -m "Add vitest setup + tests for calendar scheduling (Experiment 1.4.2)"
>> git push
warning: in the working copy of 'package-lock.json', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'package.json', LF will be replaced
  
```

7. Conclusion:

React.memo applied to the calendar's day cells, combined with useCallback-stabilized event handlers, limits renders to only the cells whose props actually changed during a drag operation. A Vitest + React Testing Library suite was set up covering both selector memoization (via referential equality checks) and real user interaction with the post composer (platform selection, validation, and draft saving), giving automated confidence in the application's core logic beyond manual UI testing.

8. Learning Outcomes:

- Learned when and why React.memo is effective, and that it requires stable prop references (via useCallback) to work.
- Learned to test memoized selectors by asserting referential equality, not just correct values.
- Learned to write component tests that simulate real user interaction rather than testing implementation details.



- Understood the distinction between manual UI testing and an automated regression-safe test suite.

Evaluation Grid (To be created as per the SOP and Assessment guidelines by the faculty):

Sr. No.	Parameters	Marks Obtained	Maximum Marks
1.	Worksheet completion		08
2.	Post Lab Quiz Result/Viva-Voce		10
3.	Conduct		12
	Total		30